

BÀI TẬP:
HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN THỜI GIAN THỰC
Nhóm 8 • K70I-CS6 • Khoa Công nghệ Thông tin • VNU-UET

PHẦN I: GIỚI THIỆU MỤC TIÊU VÀ PHẠM VI THỰC HIỆN

1. Mục tiêu đề án

- **Xây dựng nền tảng đấu giá thời gian thực:** Cung cấp hệ thống cho phép người dùng đăng bán sản phẩm và tham gia đấu giá trực tiếp. Đảm bảo tốc độ phản hồi cho các sự kiện cập nhật giá để bảo đảm tính cạnh tranh công bằng.
- **Đảm bảo tính nhất quán tài chính và giao dịch (Transaction Integrity):** Tự động hóa việc kiểm tra số dư khả dụng, thực hiện đóng băng tài sản tạm thời của người dẫn đầu phiên đấu giá, và tự động hoàn trả tức thì khi có người đặt giá cao hơn.
- **Tích hợp giải pháp tự động hóa thông minh:** Triển khai động cơ đấu giá kép hỗ trợ song song đặt giá thủ công và tự động (Auto-bid), cùng với cơ chế chống giật giải phút chót (Anti-sniping) giúp bảo vệ tối đa quyền lợi của người bán.

2. Phạm vi thực hiện

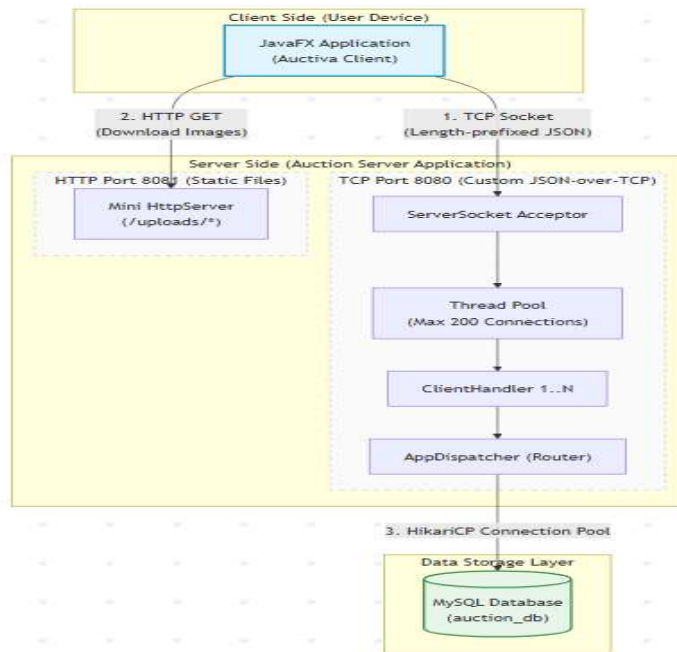
- **Hệ thống máy chủ (Auction Server):**
 - Sử dụng ngôn ngữ Java thuần túy với thư viện mạng java.net.ServerSocket hiệu năng cao, không phụ thuộc vào các Web Framework công kênh nhằm tối ưu tài nguyên phần cứng.
 - Thiết kế giao thức trao đổi điệp riêng biệt trên nền TCP Socket truyền tải chuỗi JSON có cấu trúc độ dài cố định.
 - Tích hợp Mini HTTP Server để phục vụ tải dữ liệu hình ảnh sản phẩm.
 - Cơ sở dữ liệu: Hệ quản trị cơ sở dữ liệu quan hệ MySQL kết hợp cấu trúc cột động JSON.
- **Hệ thống máy khách (Auction Client):**
 - Xây dựng ứng dụng Desktop đồ họa bằng JavaFX (MVC Pattern) đảm bảo giao diện trực quan, đa nhiệm, mượt mà.
 - Xử lý bất đồng bộ các tác vụ mạng để giải phóng hoàn toàn luồng vẽ giao diện UI.

PHẦN II: KIẾN TRÚC TỔNG THỂ VÀ MÔ TẢ HỆ THỐNG

Hệ thống tuân thủ mô hình kiến trúc phân cấp vật lý Client - Server kết hợp phân tầng logic phần mềm nhằm nâng cao tính mở rộng và bảo mật.

1. Sơ đồ Topology mạng vật lý (Deployment Topology)

Sơ đồ dưới đây mô tả cấu trúc vật lý và các giao thức mạng được sử dụng để kết nối các thành phần trong hệ thống:

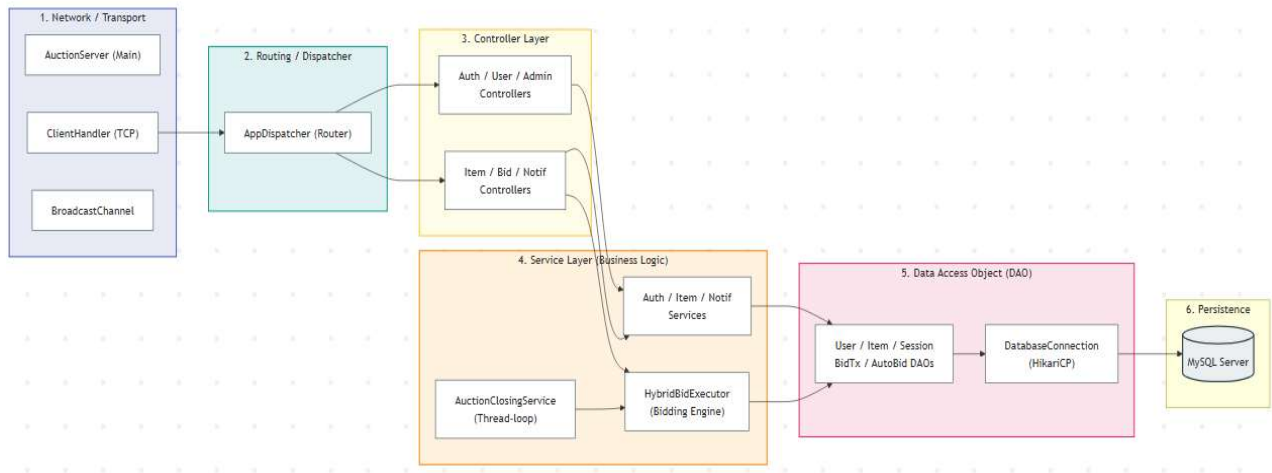


- **Cơ chế truyền thông điệp nghiệp vụ (Port 8080):**
 - Client thiết lập một kết nối TCP Socket duy nhất tới cổng 8080 của Server.
 - Để giải quyết vấn đề đóng gói/phân mảnh trong TCP Socket, dữ liệu được truyền đi dưới dạng: [4 bytes độ dài (int)] + [Chuỗi JSON].
 - Khi có gói tin đến, ClientHandler đọc 4 bytes đầu tiên để xác định kích thước chính xác, sau đó đọc đủ số bytes tiếp theo để tạo chuỗi JSON hoàn chỉnh và giải mã.
- **Cơ chế truyền tải tài nguyên tĩnh (Port 8081):**
 - Thay vì truyền tải dữ liệu nhị phân qua cổng Socket gây chiếm dụng luồng truyền tải nghiệp vụ, hệ thống chạy một Mini HTTP Server độc lập tại cổng 8081.
 - Client tải ảnh trực tiếp bằng các truy vấn HTTP thông thường (ví dụ: <http://localhost:8081/uploads/dienthoai.png>).

2. Kiến trúc phân tầng phần mềm của Server (Layered Software Architecture)

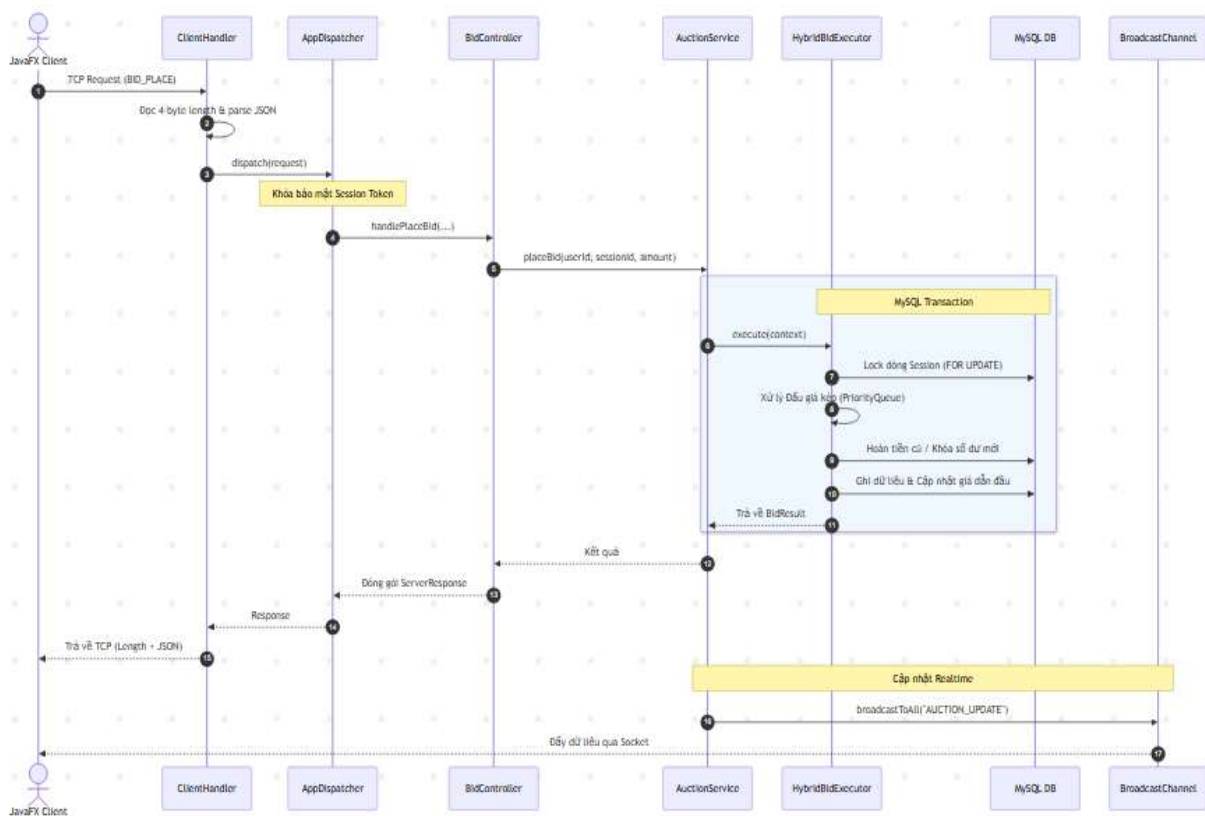
Server được thiết kế theo kiến trúc phân tầng kinh điển giúp tách biệt rõ ràng trách nhiệm giữa các thành phần logic:

- **Tầng Network (Lớp 1):** Tiếp nhận kết nối vật lý. ClientHandler.java quản lý luồng vào/ra của dữ liệu mạng và đồng bộ trạng thái trực tuyến của User thông qua BroadcastChannel.java.
- **Tầng Routing (Lớp 2):** AppDispatcher.java đóng vai trò cổng bảo mật. Nó giải mã gói tin JSON, kiểm tra tính hợp lệ của phiên đăng nhập và điều phối luồng xử lý tới Controller phù hợp.
- **Tầng Controller (Lớp 3):** Nhận yêu cầu đã xác thực từ Router, bóc tách tham số chi tiết và chuyển giao việc xử lý nghiệp vụ xuống tầng Service.
- **Tầng Service (Lớp 4):** Nơi thực thi toàn bộ logic nghiệp vụ thực tế. Động cơ tính toán giá thầu HybridBidExecutor.java xử lý các xung đột giá của người chơi và AuctionClosingService.java tự động quét đóng phiên.
- **Tầng DAO (Lớp 5):** Đóng gói mã lệnh SQL thô, sử dụng Connection Pool thông qua DatabaseConnection.java nhằm duy trì hiệu năng kết nối cơ sở dữ liệu ở trạng thái tối ưu nhất.



3. Sơ đồ tuần tự tương tác động (Bidding Event Sequence Diagram)

Sơ đồ dưới đây mô tả chi tiết đường đi của luồng thông tin và hoạt động đồng bộ dữ liệu khi một Client gửi yêu cầu đặt giá (Bid) lên hệ thống:



PHẦN III: BÁO CÁO CHỨC NĂNG THEO BAREM ĐIỂM

HẠNG MỤC 1: THIẾT KẾ LỚP VÀ CÂY KẾ THỪA (2.5đ)

1. Xác định và triển khai các lớp chính

Hệ thống tổ chức gói chung auction-common làm nơi định nghĩa các thực thể cốt lõi sử dụng chung cho cả Client và Server:

- **User:** Đại diện cho người dùng hệ thống. Chứa các trường ID, thông tin cá nhân, mật khẩu băm, quyền hạn và số dư tài chính (balance, frozen_balance).
- **Item:** Mô hình hóa sản phẩm đấu giá. Quản lý người bán (sellerId), thông tin sản phẩm và trường Specs động.
- **AuctionSession:** Đại diện cho phiên đấu giá đang diễn ra, lưu trữ giá khởi điểm, giá hiện tại, thời gian bắt đầu và kết thúc.
- **BidTransaction:** Lưu trữ lịch sử tất cả các lượt trả giá để phục vụ tra cứu và kiểm toán.
- **AutoBidConfig:** Cấu hình các lượt đặt giá tự động của người mua.

2. Áp dụng đúng các nguyên tắc OOP (1.0đ — Bắt buộc)

- **Encapsulation:** Tất cả các thuộc tính dữ liệu trong Entity và DTO đều được thiết lập phạm vi truy cập private. Việc tương tác dữ liệu bắt buộc phải đi qua các hàm getter/setter và Builder Pattern.
- **Abstraction:** Định nghĩa các Interface để ẩn giấu chi tiết cài đặt phía sau, tiêu biểu là BroadcastChannel. Người dùng chỉ gọi phương thức gửi tin mà không cần biết cách Socket quản lý danh sách tuyến dưới.
- **Inheritance:** Áp dụng trong cây phân cấp ngoại lệ, xây dựng lớp cơ sở BusinessException, đó kế thừa ra ValidationException, giúp quản lý và xử lý lỗi tập trung.
- **Polymorphism:** Thể hiện rõ trong cách AppDispatcher ánh xạ và chuyển đổi dữ liệu Response thành các định dạng DTO khác nhau tùy theo cấu hình đa hình của GSON serializer.

3. Áp dụng Design Pattern phù hợp (1.0đ — Bắt buộc)

Hệ thống sử dụng linh hoạt các mẫu thiết kế kinh điển:

- **Singleton Pattern:** Quản lý Connection Pool tại DatabaseConnection.
- **Observer Pattern:** AuctionEventBus và Subscriber phát tín hiệu đổi giá bất đồng bộ.
- **Builder Pattern:** Dùng thư viện Lombok @Builder cho Entity/DTO tránh quá tải Constructor.
- **DAO Pattern (Data Access Object):** Tách biệt toàn bộ mã SQL ra khỏi Service chính

HẠNG MỤC 2: CHỨC NĂNG CHÍNH (3.0đ)

1. Quản lý người dùng, sản phẩm (1.0đ — Bắt buộc)

- **Quản lý người dùng:** Thực hiện các chức năng Đăng ký, Đăng nhập bảo mật cao, Cập nhật thông tin cá nhân, Nạp tiền/Rút tiền vào ví ảo cá nhân.
- **Quản lý sản phẩm:** Cho phép người bán đăng sản phẩm mới. Hệ thống cho phép các thuộc tính chung (Tên, Ảnh) lưu ở cột tĩnh.

2. Chức năng đấu giá (1.0đ — Bắt buộc)

- Hệ thống có tạo các phiên đấu giá công khai với giá khởi điểm và thời hạn kết thúc xác định.
- Bidders gửi yêu cầu đặt giá trực tiếp. Hệ thống tính toán bước giá tối thiểu hợp lệ kế tiếp và cập nhật ngay lập tức trạng thái phiên thầu lên màn hình JavaFX Client.

3. Xử lý lỗi & Ngoại lệ (1.0đ — Bắt buộc)

- **Phía Server:** Sử dụng cơ chế bọc kiểm tra dữ liệu nghiêm ngặt (ValidationException). Nếu xảy ra lỗi nghiệp vụ (ví dụ: Đặt giá thấp hơn bước giá tối thiểu, Không đủ số dư), hệ thống sẽ ném ngoại lệ và tự động thực hiện Transaction Rollback để trả cơ sở dữ liệu về trạng thái an toàn.
- **Phía Client:** Thiết kế lớp AlertUtil để hiển thị hộp thoại cảnh báo thân thiện cho người dùng. Nếu kết nối Socket bị ngắt đột ngột, Client sẽ ngắt kết nối an toàn và hiển thị thông báo yêu cầu kết nối lại mà không làm crash ứng dụng.

HẠNG MỤC 3: KỸ THUẬT QUAN TRỌNG & CONCURRENCY (1.5đ)

1. Xử lý đấu giá đồng thời an toàn (1.0đ — Bắt buộc)

- Để giải quyết xung đột khi nhiều người cùng đặt giá vào cùng một mili-giây, hệ thống thực thi khóa dòng phiên đấu giá trong SQL: `SELECT * FROM auction_session WHERE session_id = ? FOR UPDATE`.
- Lệnh thầu cạnh tranh sẽ bị chặn và xếp hàng đợi cho đến khi Transaction trước đó hoàn tất. Điều này giúp triệt tiêu hoàn toàn các lỗi cập nhật bị mất và tình trạng bán khống/hoàn tiền sai lệch.

2. Realtime Update (0.5đ — Bắt buộc)

- Khi có bất kỳ giao dịch đặt giá thành công nào được ghi nhận tại `HybridBidExecutor.java`, Server sẽ gửi tín hiệu thông báo qua `BroadcastChannel`.
- Kênh quảng bá sẽ gửi thông điệp tới toàn bộ các kết nối Socket đang hoạt động của Client để cập nhật tức thời giao diện giá thầu mà không yêu cầu Client tải lại trang.

HẠNG MỤC 4: TÍCH HỢP, KIẾN TRÚC & CHẤT LƯỢNG MÃ (2.5đ)

1. Thiết kế kiến trúc Client-Server rõ ràng (0.5đ)

- Sử dụng giao thức tự định nghĩa trên nền TCP Socket. Bản tin truyền đi chứa 4 bytes chỉ định độ dài chuỗi để đảm bảo dữ liệu không bị dính gói.
- Các chức năng gửi/nhận yêu cầu được chuẩn hóa dưới dạng JSON đồng bộ giữa Client và Server.

2. Áp dụng MVC (0.5đ)

- **Phía Client:** Ứng dụng cấu trúc MVC của JavaFX. Giao diện thiết kế bằng file `.fxml` (View), quản lý logic hiển thị và thu thập dữ liệu bằng các lớp Controller (như `AuctionDetailController`), tương tác dữ liệu qua Model DTO.
- **Phía Server:** Thiết kế Controller - Service - DAO phân tách rõ ràng trách nhiệm xử lý logic.

3. Sử dụng Maven, coding convention tốt, mã nguồn sạch (0.5đ)

- Dự án quản lý tập trung bằng Maven đa mô-đun (Multi-module) cấu hình trong `pom.xml`.
- Áp dụng chuẩn Google Java Style Guide bằng cách tích hợp Plugin Checkstyle `google_checks.xml`, buộc mã nguồn phải luôn sạch sẽ, không thừa biến và tuân thủ quy tắc đặt tên.

4. Unit Test cho logic quan trọng (0.5đ)

- Viết đầy đủ các ca kiểm thử tự động (Unit Test) sử dụng JUnit 5 cho các phần xử lý quan trọng trong thư mục `src/test/java`, bao gồm kiểm thử logic Đấu giá tự động (`AutoBidServiceTest`), Cập nhật sản phẩm (`ItemWriteServiceTest`), và Hoạt động mạng (`BroadcastChannelImplTest`).

5. Thiết lập CI/CD cơ bản (0.5đ)

- Thiết lập GitHub Actions thông qua file `.github/workflows` tự động kích hoạt tiến trình Build Maven và chạy toàn bộ Unit Test mỗi khi có hành động Push hoặc Pull Request lên nhánh chính.

HẠNG MỤC 5: CÁC CHỨC NĂNG NÂNG CAO (Đạt tối đa 1.5đ)

1. Auto-Bidding — Đấu giá tự động sử dụng PriorityQueue (0.5đ)

- Khi người dùng kích hoạt Auto-bid với một mức giá trần (`maxPrice`), hệ thống sẽ tự động đặt giá thay họ mỗi khi bị vượt mặt.
- Để giải quyết trường hợp nhiều người cùng cài đặt Auto-bid trong một phiên đấu giá, hệ thống sử dụng cấu trúc dữ liệu PriorityQueue chứa các đối tượng `BidCommand`. Hệ thống sẽ so khớp chênh

lệch giữa các mức giá trần để tự động tăng giá trị phiên thầu lên mức tối thiểu đủ để dẫn đầu, giúp tối đa hóa khả năng chiến thắng với chi phí tối ưu nhất cho người dùng.

2. Gia hạn phiên đấu giá — Anti-Sniping (0.5đ)

- Để ngăn chặn hành vi sử dụng bot bắn giá ở giây cuối cùng của phiên đấu giá, lớp AntiSnipingService sẽ kiểm tra thời gian nhận thầu.
- Nếu phát hiện lượt đặt giá hợp lệ xuất hiện trong vòng 30 giây cuối cùng trước giờ đóng thầu, hệ thống sẽ tự động cập nhật thời gian kết thúc của phiên thầu kéo dài thêm 30 giây nữa, tạo cơ hội cạnh tranh công bằng cho tất cả mọi người.

3. Trực quan hóa lịch sử đặt giá bằng Biểu đồ thời gian thực (0.5đ)

- Trong màn hình chi tiết phiên đấu giá của ứng dụng Client JavaFX, hệ thống tích hợp một cấu phần biểu đồ đường.
- Biểu đồ này tự động vẽ và cập nhật trực quan biến động của các mức giá thầu theo dòng thời gian mỗi khi nhận được sự kiện Socket cập nhật từ Server, giúp người dùng nắm bắt xu hướng đấu giá một cách trực quan nhất.